TECHNICAL / PROTOCOL WHITE PAPER

A private, on-chain reservation primitive

The architecture, the NEAR contract, and the privacy model of a reservation registry that proves *who reserved what* without writing prices, terms, or personal data to the chain — written for protocol engineers.

✓ VETTED NETWORK

Document
Technical / protocol edition

Status
Testnet · calculating-hand.testnet

Contract
Rust · near-sdk 5.26.1 · state V1→V7

As of
November 2026

SCOPE & HONESTY

This edition goes deep on architecture, the contract's state machine and migrations, the public privacy model, and the method surface. Status language is literal: **built** runs today; **in progress / planned** is designed or partially built but not yet usable as described. Facts are taken from the real contract source and the standing security audit — nothing here is aspirational unless it is tagged as such.

ABSTRACT

In one page

VRX is a reservation primitive: a callable NEAR smart contract that records *who reserved what*, paired with an off-chain portal that carries everything the chain must not — identities, prices, specifications and deal terms.

An operator with **unique, non-substitutable inventory** (unit 312 is not unit 311) needs to feed live availability to many sales teams and agencies who do not trust one another, and let them browse, book, reserve, take a deposit and hold — without double-booking, without leaking the commercial or personal details of a deal, and without a channel being able to go around the operator. A public chain gives tamper-evidence and independent verifiability but, used naïvely, publishes the very terms that make a deal confidential. A single vendor database gives confidentiality but centralizes trust and sees everything.

VRX's design answer is a strict **split**. The chain holds only opaque hashes and neutral account ids: it proves *that* a reservation exists and *who* holds it, and nothing else. The meaning — the *what*, the *how-much*, the *with-whom* — lives off-chain under server-side authorization and field-level masking. Privacy is therefore not a feature bolted on; it is a property of *what is and is not written to the ledger*.

> **WHY IT IS DOMAIN-AGNOSTIC**
>
> Real estate (off-plan property) is the beachhead and the working proof. But the contract never "knows" it is real estate — every real-world reference is an opaque SHA-256 hash — so the same primitive serves cars, livestock, equipment, event slots: any Class-A, non-substitutable inventory. The contract models *availability and holds over keys it cannot read*.

ON-CHAIN	Reservation / hold / exclusive / attestation registry — opaque hashes + neutral ids only.
OFF-CHAIN	The portal: role-scoped apps, sealed records, field masking, consent ceremonies.
CONTRACT	Rust / near-sdk 5.26.1, `#[near]` macros; state schema V1 → V7 via borsh-safe migrations.
EVENTS	NEP-297 structured `EVENT_JSON` logs (standard `vrx`, v1.0.0) — opaque payloads only.
CREDIT	$VRX — fixed 200M, treasury-held; off-chain ledger today, NEP-141 planned.
POSTURE	VRX facilitates and seals; it does not adjudicate disputes or warrant enforceability.

SECTION 1

The problem

Selling scarce, one-of-a-kind inventory through many channels that do not trust each other breaks the tools everyone already uses. Three failures compound.

1 — Double-booking a unique thing

When inventory is fungible (seats, generic SKUs) an oversell is an inconvenience. When each item is unique — a specific apartment, a specific pedigree animal, a specific machine — two channels reserving "312" at once is a hard conflict that damages trust and, often, a contract. The market needs **one authoritative answer** to "is this exact unit still available?" that every channel reads the same way, in real time.

2 — Privacy leakage

The naïve fixes leak. A shared spreadsheet or a single vendor database exposes who is buying what, at what price, through whom — to the vendor and often to rival channels. Putting the registry on a public blockchain is worse: now prices, parties and terms are world-readable forever. The commercially-sensitive and personal details of a deal are exactly what must *not* be shared to get the shared truth.

3 — Circumvention / disintermediation

A channel that learns the end-buyer and the operator can be tempted to cut the introducer out and deal direct. The counter-measure — hiding the buyer's identity until a deal is firm — is itself a privacy problem, and it must be reconciled with the operator's legitimate need to confirm a booking.

> **THE TENSION IN ONE LINE**
>
> Everyone needs to agree on *the same reservation state*, while almost no one is entitled to see *the same reservation details*. A single store that satisfies the first requirement usually violates the second. VRX exists to hold both at once — and the whole architecture is a consequence of that single constraint.

Who feels it first

Off-plan real estate is the sharpest case and VRX's beachhead: a developer releases limited units to multiple agencies, each with multiple agents, selling to buyers who must stay confidential — with deposits, commissions and completion terms attached. Every failure above is routine there. The same shape recurs anywhere inventory is scarce, identified, and sold through intermediaries — which is why the on-chain layer is written to know nothing about real estate at all.

SECTION 2

What VRX is

A callable NEAR smart contract — the reservation and hold registry — plus an off-chain portal: the role-scoped apps each participant uses. Together they are the reservation primitive.

The primitive, precisely

VRX records reservations over **non-substitutable** inventory. Its on-chain state is deliberately thin. A unit (a `tile`) is `open`, on `hold`, `closed`, or `done` — encoded as the small integers `0/1/2/3`. A `claim` is open, accepted, declined, offered-an-alternative, or confirmed (`0/1/2/3/4`). An exclusive window is held by one agency or it is not. Everything that would identify the unit, the buyer, the price or the terms is an opaque hash. The contract does not model "apartment" or "car" — it models availability and holds over keys it cannot read.

> `312 ≠ 311`
>
> Value in this system scales with the worth of the *specific* reserved item. The primitive's whole reason to exist is that substituting one unit for another is not acceptable — so the registry tracks each unit individually, keyed by an opaque `unit_key`, and a hold on one is never a hold on another.

The four pillars, wrapped in vetting

PILLAR	WHAT IT IS	STATUS
Availability	The foundation — what inventory exists and is open.	**built**
Communications	Availability made useful: conveyed to the market, with channels + change-notice.	**built**
Payments	Communication turned into action — deposits, settlement projection.	**built / rail planned**
Accounting	Every payment tracked (two-sided attestation ledger; VRX does not move money in-app).	**built**

Vetting ties the four together: VRX vets the whole system so participants who do not trust each other can transact on one shared truth. Vetting is human-led (see §6); the chain only anchors the *fact and version* of a sign-off, never its contents.

ON-CHAIN	reservation / hold / exclusive / attestation state (opaque)
OFF-CHAIN	the four pillars' working surfaces + all meaning
BINDING	every on-chain record anchors an off-chain sealed record by its hash

SECTION 2.5

Where it sits

Why nothing already in the market does this. Stated plainly — the moat is a design choice others have not made, not a claim of being uniquely clever.

Three existing approaches, and where each leaks

APPROACH	GIVES YOU	WHAT IT COSTS
Generic booking / CRM software	A shared calendar and pipeline.	Centralizes trust in one vendor's database; the vendor (and often rival channels) can see who buys what at what price. No independently-provable record; the operator still owns the single point of truth and failure.
Naïve on-chain listing	A tamper-evident, independently-verifiable record.	Publishes prices, parties and terms to the world forever. The confidentiality that makes a deal a deal is destroyed to gain the shared truth.
Point-to-point / off-platform deals	Privacy by obscurity.	No shared availability, so double-booking and circumvention are endemic; no neutral record when a dispute arises.

VRX'S POSITION

VRX takes the tamper-evident, independently-verifiable record from the on-chain approach and the confidentiality from the off-platform approach, and reconciles them with a **split**: the chain carries proof-of-state over opaque keys; a private, access-controlled off-chain layer carries the meaning. The moat is not a faster database — it is the discipline of *what is never written to the ledger at all*, plus a vetted operator network layered on top.

Honest limits of the claim

- ✓ The off-chain layer's confidentiality rests on **server-side authorization**, not cryptography — VRX must be trusted to run it correctly. The audit (§5) is how that trust is made checkable.
- ✓ On-chain opacity is absolute by construction, but it only protects what lives on-chain; a leak, if it happened, would come from the off-chain layer — which is where almost all the engineering attention goes.
- ✓ VRX is not the only system splitting on/off chain. Its specific contribution is the reservation-primitive framing, the vetted network, and a provenance/forensic layer — not the generic idea of anchoring a hash.

SECTION 3 · ARCHITECTURE

The on-chain / off-chain split

One rule governs the whole system: **meaning never touches the chain**. The figure shows what lives where, and the one-way binding between them.

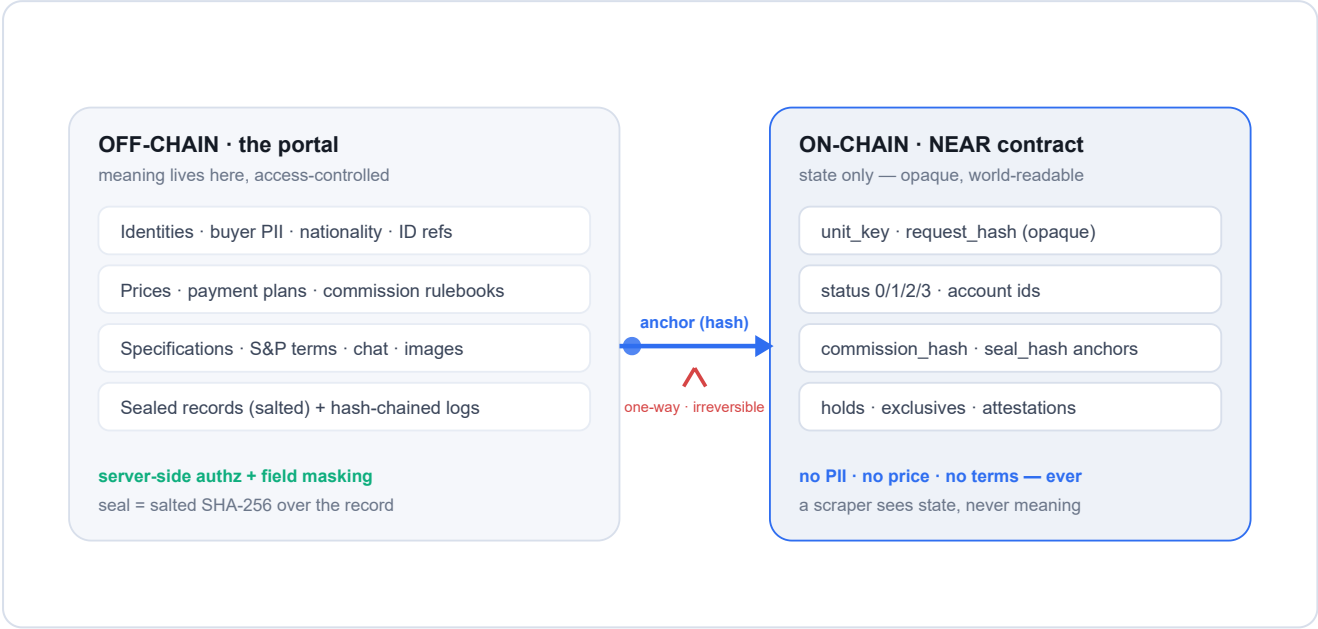


Figure 1 — The split. Off-chain records are sealed; only their hashes are anchored on-chain. The binding is one-way.

WHY ONE-WAY MATTERS

An anchored `seal_hash` lets anyone later prove a specific off-chain record is the one that was committed (re-seal it, compare). It never lets anyone recover the record from the hash. Tamper-evidence without disclosure is the whole trick — and it is the property every design decision downstream protects.

SECTION 3 · ARCHITECTURE

Contract state & migrations

The contract (Rust, near-sdk 5.26.1, `#[near]` macros) has grown through seven state schema versions, each added by an owner-only, borsh-safe migration that never touches an existing record.

What each version added

VERSION	ADDED	SHAPE
V1	Core registry — nodes, orgs, peers, gates, tiles, claims, queue.	base maps
V1→V2	Booking-pass rebirth lineage.	`pass_links` sidecar
V2→V3	On-chain hold expiry + a configurable hold window.	`hold_expiry` sidecar + `hold_window_ns`
V3→V4	Commission `agreements` (anchor only).	`agreements` map
V4→V5	Protocol usage toll + treasury + exempt list.	`toll_yocto` · `treasury` · `exempt`
V5→V6	Temporary exclusives (sole-right-to-book window).	`exclusives` map
V6→V7	Vetting / sign-off attestations (on-chain provable approval).	`attestations` map

The borsh-safe migration pattern

Borsh is order-based and ignores field names, so two rules keep migrations safe. First: **never append a field to a struct that is stored as a value** — that would shift the byte layout and break decode on every old entry. Instead, extra per-key data lives in a **sidecar map** (that is what `pass_links`, `hold_expiry` and `exclusives` are). Second: each migration reads the prior versioned struct, *carries every existing map over unchanged*, and only creates the one new top-level map empty. Each is owner-gated — the caller must equal the `owner` recorded in the pre-migration state — and emits a `Migrate` event.

```
// migrate7 (V6 -> V7): add the empty `attestations` map; carry everything else over.
// Owner-only, pure-additive, borsh-safe — no existing record is touched.
#[init(ignore_state)]
pub fn migrate7() -> Self {
    let v6: ContractV6 = env::state_read().expect(err::NO_V6_STATE);
    assert_eq!(env::predecessor_account_id(), v6.owner, "{}", err::NOT_OWNER);
    Event::Migrate { from: 6, to: 7 }.emit();
    Self {
        owner: v6.owner, nodes: v6.nodes, orgs: v6.orgs, peers: v6.peers,
        gates: v6.gates, tiles: v6.tiles, claims: v6.claims, queue: v6.queue,
        /* ...passes, pass_links, hold_expiry, agreements, toll, treasury, exempt,
exclusives... */
        attestations: UnorderedMap::new(SK::At),  // the only new map, created empty
    }
}
```

FIELD-NAME DISCIPLINE

Source field names are readable; each pins its original compact JSON key via `#[serde(rename)]`,

SECTION 3 · ARCHITECTURE

The opaque data shapes

What the contract actually stores. Every real-world identifier is a `String` that holds a hash; the contract treats them as opaque keys and never interprets them.

The top-level maps (V7)

MAP	KEY	CARRIES
`nodes`	account id	a seller/operator + its `instance_hash`
`orgs`	account id	an agency + an opaque `org_ref`
`peers`	account id	an agent + an opaque `agent_ref`, scoped to an org
`gates`	`"{node}:{org}"`	access scope = a set of opaque scope hashes
`tiles`	`unit_key` (hash)	status `0/1/2/3` + an `integrity_hash`
`claims`	`request_id`	status `0/1/2/3/4` + `request_hash` anchors
`agreements`	`"{node}:{org}"`	an opaque `commission_hash` + a version
`exclusives`	`unit_key`	which org holds the sole-book window, and until when
`hold_expiry`	`unit_key`	hold expiry timestamp (ns); absent = no timed hold
`attestations`	`"{node}:{building_hash}"`	a sealed sign-off anchor (`seal_hash`, kind, approver)
`passes`	pass id (u64)	a non-transferable booking token + its owner / lineage

THE INVARIANT THAT MAKES IT DOMAIN-AGNOSTIC

Note what is *absent*: no field names like "price", "buyer", "address", "bedrooms". The storage layer has no vocabulary for real estate — only hashes, account ids, small status integers, timestamps and versions. The same contract, unchanged, backs a car dealership or a livestock registry; the meaning of a `unit_key` is decided entirely off-chain.

```
// An attestation anchors a sealed off-chain sign-off — the ONLY content bytes are a hash.
pub fn attest(building_hash: String, kind: u8,
              seal_hash: String, approved_by: AccountId, approver_role: u8) {
    // no price, no name, no terms — just the opaque proof the sign-off happened,
    // keyed "{node}:{building_hash}". VRX later ratifies it (ratify_attestation).
}
```

SECTION 3 · ARCHITECTURE

Data flow, end to end

Follow one booking from live availability to a confirmed hold, watching where meaning stays and only a hash moves.

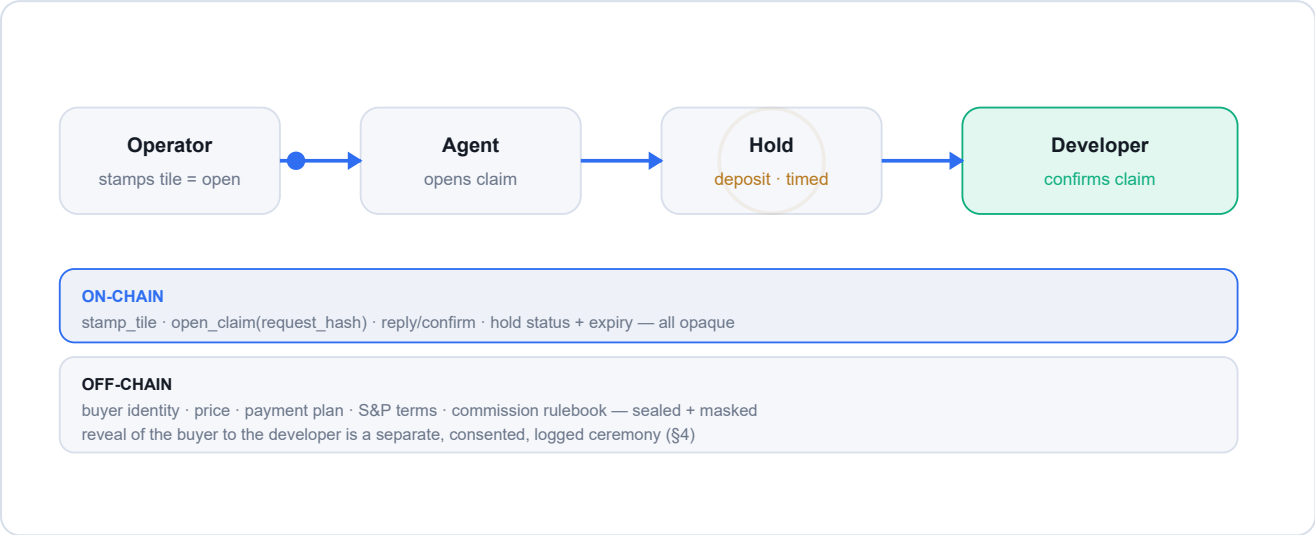


Figure 2 — One booking, end to end. Only opaque keys + state cross into the on-chain lane.

The step sequence

1. Operator `stamp_tile(unit_key, 0, integrity_hash)` → the unit is `open`.
2. Agent `open_claim` (or `burn_pass_hold` on the booking-token path) → an opaque `request_hash` anchors the off-chain request; the unit moves to `hold` with a timed expiry in `hold_expiry`.
3. Off-chain: deposit recorded, buyer profile sealed, masked to the developer by default.
4. Developer `reply_claim` / `confirm_claim` → claim status advances; a consented reveal of the buyer is a separate ceremony.
5. On completion `finalize_tile` → `done`; an expired hold auto-releases back to `open` (via `release` / expiry).

SECTION 4 · PRIVACY

The privacy model

Two distinct protections, applied per surface: on-chain opacity by construction, and off-chain access control by the server. This section is the public, capability-and-design level of the model.

Protection 1 — on-chain opacity (by construction)

Everything on the contract is an opaque SHA-256 hash (`unit_key` , `request_hash` , `seal_hash` , `commission_hash`) plus neutral account ids and status integers. A chain scraper sees *"account X holds an exclusive on hash 07f2…"* — state, never meaning. The building, unit, price, party and terms cannot be derived from the chain. This moat holds by design and needs no further work.

The seal construction (public part)

A **seal** is a salted SHA-256 over an off-chain record. The salt is per-record; an optional global *pepper* is held separately from the record store, so a breach that leaks the store alone cannot re-derive seals. A seal is a commitment, not an encryption key — it proves a specific record is the one that was committed (re-seal and compare) and reveals nothing about the record's contents. Where a real-world identifier must be a *stable* opaque key (so two references to the same thing collide deterministically), the key is an **HMAC** under a server-held key rather than a bare hash — same opacity, with unforgeability against an outsider who does not hold the key.

Protection 2 — off-chain access control (by the server)

The plaintext that carries meaning lives off-chain, protected by **server-side authorization + field-level masking**, not by cryptography. The engineering question for every counterparty-facing surface is therefore: *is this read scoped to a caller entitled to it, and masked server-side before the wire?* A UI blur is theatre; masking happens on the server, so the plaintext never reaches the browser for a masked field.

Supporting mechanisms (built)

- ✓ **Salted seals** (`lib/seal.ts`) — tamper-evidence without reversibility, as above.
- ✓ **Encryption-at-rest** for identity-document images — a per-file data-encryption key under an offline-sealed key-encryption key.
- ✓ **Append-only, hash-chained chat** (`lib/channels.ts`) — each message folds the prior record's hash into its own, so editing or reordering a past message breaks the chain and is detectable.
- ✓ **Provenance-stamped imagery** — a leaked sensitive image is traceable back to the recipient it was served to. The stamping mechanism is deliberately private and is not described in this paper.

What an outsider can and cannot see

OBSERVER	CAN SEE	CANNOT SEE
Chain scraper	That a reservation/hold/exclusive exists; which account holds it; status + version.	Unit, building, price, party, terms — anything meaningful.
A rival channel	Shared availability (open/held).	Another channel's buyer, price or terms.
Authority-review role	The *vetted state* for oversight.	Private buyer data — the moat extends to the regulator view.

SECTION 5 · AUDIT

Threat model & invariants

The bar VRX judges itself against, drawn from the standing security/privacy audit. The method: test the *promises*, hunt *contradictions* at the seams. These are the checks an external auditor should reproduce.

Buyer-PII surfaces (the core map)

SURFACE	ENCRYPTED AT REST?	MASKED TO COUNTERPARTY?	READS PLAINTEXT
A. Agent roster `lib/buyers.ts`	sealed only	never cross-shown	owning agent only
B. Booking-offer buyer `lib/offers.ts`	sealed only	yes, until a consented reveal	agent full; developer masked
C. KYC vault `lib/kyc-vault`	images yes; fields **deferred**	party-scoped decrypt at the final gate	scoped reader at pre-sign gate
D. Direct-sales buyer `lib/buyers-direct.ts`	sealed only	yes	buyer self full; desk masked

The seven invariants an auditor should test

1. **No PII or plaintext terms on-chain** — only opaque hashes + neutral ids.
2. **Every counterparty-facing buyer projection is masked server-side** — the plaintext never reaches the browser for a masked field.
3. **Every reveal is a witnessed, sealed, consented act** — never a silent flip; the sealed declaration is on the trail even on decline.
4. **The chat is append-only + hash-chained** — editing / reordering / deleting a past message is detectable and prevented.
5. **A seal re-verifies iff the plaintext is unaltered** — tamper or a lost salt returns false, never a silent pass.
6. **Self-read full, counterparty-read masked** — on every role route.
7. **No silent on-contract change** — every change to a partnered building seals a before/after evidence record and notifies affected parties; the most sensitive particulars change only via a VRX-reviewed correction.

> **THE ONE DELIBERATE INVERSION (AN AUDITOR MUST UNDERSTAND THIS)**
>
> The system masks a buyer to a counterparty by default everywhere. Exactly one place inverts: the **prior-block force-show**. When a desk hard-blocks a buyer, that buyer's full identity is thereafter force-shown *to that desk only* — so a blocked bad actor cannot re-approach the same desk anonymously. It is desk-scoped (never global), triggered only by a deliberate block, timestamped, and applied server-side in that desk's projection alone. An intentional, scoped, logged anti-cheat exception — not a leak.

SECTION 6

Vetting & the operator network

VRX is not only a contract — it is the vetting entity and the network operator. Identities enter the system only through VRX; the chain then anchors the fact of a sign-off.

Owner-gated onboarding

Provisioning a node (seller) or org (agency) is **owner-only**: the contract owner is the vetting identity. KYC / vetting is a human-led off-chain precondition done *before* the on-chain `register_node` / `register_org` call — nothing identifying ever lands on-chain. A developer's captive in-house sales team is simply an org provisioned the same way; there is no special path. The same owner-gating protects the admin surface (`set_toll`, `set_treasury`, the exempt list) and every migration.

The eight participant roles

ROLE	DOES
Developer(s)	Own the inventory; grant access, mark availability, confirm bookings; may sell direct.
Direct-seller	The developer's own desk selling to a walk-in buyer (not a separate actor).
Agent(s)	Reserve units for buyers on behalf of an agency.
Agency/ies	Manage their agents; work the developers who grant them access.
Buyer	Browses direct-sale inventory and transacts via the desk — a *state*, not a vetted role (any wallet).
Direct-buyer	A walk-in going straight to a desk; on the real path, meets a light anti-abuse eligibility bar **(planned)**.
CEO / Approver	Oversees high-stakes financial accountability (an approval trust-dial VRX sets per developer).
Authority Review	Sees the *vetted state* for oversight without ever seeing private data.

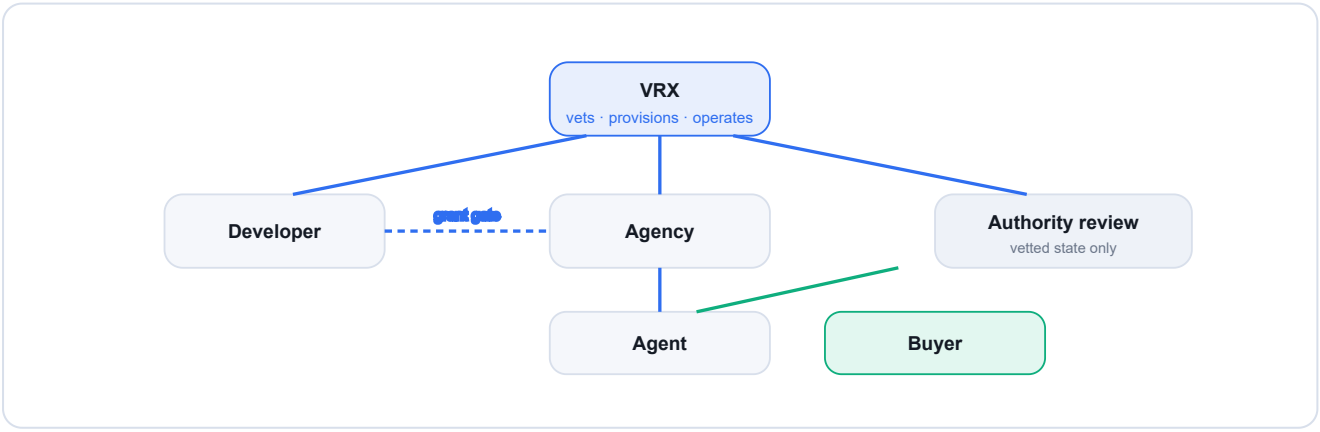


Figure 3 — Roles & trust network. VRX vets the whole system; developers gate agencies; agencies supervise agents; agents serve buyers.

SECTION 6

Non-circumvention & booking tokens

Two mechanisms keep the network honest: the buyer stays masked until a deal is firm (so a channel cannot be cut out), and reservation capacity is a scarce, tamper-evident token that cascades down the trust chain.

Non-circumvention

Trust is enforced by **visibility**, not disclosure. An agent holding a booking has the unit off-market; the developer and agency see *who holds it* but not necessarily the private buyer behind it until a consented, sealed reveal (§4). A non-circumvention declaration is sealed as evidence at the reveal request — even if the reveal is later declined — so the introduction is on the tamper-evident trail.

The booking-token cascade

VRX issues a developer a scarce quantity of **booking tokens** — reservation capacity. These are handed or sold down the trust chain (developer → agency → agent), with a tamper-evident receipt at each hand-off. An agent holding a token can secure a booking and take a deposit. Capacity is scarce *and* reusable: a released or cancelled token returns up the chain. Tokens surface as "coins" in the wallet.

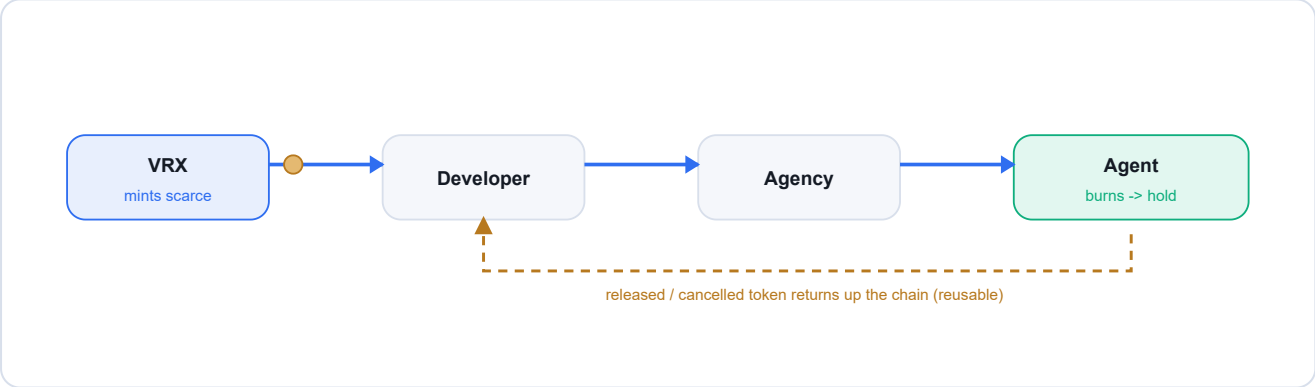


Figure 4 — Booking-token cascade. Scarce, reusable, tamper-evident at each hand-off; a burn opens a hold on-chain.

ON-CHAIN SUPPORT

The contract backs this with a non-transferable `Pass` (`mint_passes` , owner → agent), `burn_pass_hold` (spend a pass to open an instant hold), and `rebirth_pass` (a burned pass returns as a fresh one, its lineage recorded in the `pass_links` sidecar). The exclusive-hold primitive (`grant_exclusive`) grants one agency the sole right to book a unit for a bounded window, auto-reverting to the shared pool at expiry (`release_exclusive_if_expired`).

SECTION 7

The economic layer

Three economic objects: the $VRX credit ledger, booking tokens as reservation capacity, and commission agreements with a settlement projection. All privacy-preserving; none puts money terms on-chain.

$VRX — the in-system credit

SUPPLY	Fixed 200,000,000 — non-inflationary, treasury-held.
OPERATIONS	Claim from treasury · send to another account · spend inside the flow.
FORM TODAY	**built** — an off-chain ledger (`lib/vrx-token.ts`), usable within VRX. Not fiat, not redeemable for cash.
PLANNED	**planned** — the on-chain NEP-141 fungible-token form.

Booking tokens — reservation capacity

Distinct from $VRX: a booking token is a unit of *capacity to reserve*, not a credit. It is scarce, cascades down the trust chain, and returns on release (§6). On-chain it is the non-transferable `Pass` with burn + rebirth.

Commissions & settlement projection

A commission `Agreement` is keyed `"{node}:{org}"` — the same key as the access gate, so an org↔node relationship has two parallel on-chain records: the gate (access scope) and the agreement (commission). The on-chain record holds **only an opaque HMAC anchor** (`commission_hash`) of the off-chain commission rulebook plus a version; the actual percentages, tiers, splits and amounts never appear on-chain. Money compute and owed→paid tracking stay off-chain (anchor-only). On-rail settlement is **planned**; VRX today records payments as a two-sided attestation ledger and **does not move money in-app**.

How values are handled (stated plainly)

- ✓ Prices, plans, splits and amounts are computed and stored **off-chain only**; the chain carries a versioned hash anchor, never a number.
- ✓ A commission rulebook is **versioned**: changing it writes a new version + a new anchor, so a settlement projection always references a specific sealed rulebook version rather than a mutable "current" value.
- ✓ Rounding and currency handling are an off-chain concern (tested under `lib/price-rounding.ts` and the settlement-projection suite); the contract never performs money arithmetic.
- ✓ The protocol usage toll (`toll_yocto`) is the only value the contract itself moves — a flat, owner-set amount in yoctoNEAR forwarded to the treasury, waived for exempt callers. It is unrelated to deal terms.

SECTION 8

Legal posture

The posture is structural and deliberately narrow: VRX facilitates and seals; it does not adjudicate and does not warrant enforceability.

What VRX is, legally

- ✓ **A facilitator and a sealer.** VRX provides the private shared truth and the tamper-evident record. It records that a reservation, agreement or change happened, and seals it.
- ✓ **Not an adjudicator.** VRX does not decide disputes and does not determine the outcome of a disagreement between parties. It records the evidence (before/after, who was notified, the sealed declarations) — never the dispute's resolution.
- ✓ **Not a warrantor of enforceability.** VRX does not warrant that any underlying contract is legally enforceable. Parties' own terms and local law govern.
- ✓ **Privacy-by-design.** The architecture itself (opaque on-chain, masked off-chain) is the primary privacy control, not a policy promise layered on afterwards.

> **WHY THE NARROWNESS IS A FEATURE**
>
> By refusing to adjudicate or warrant, VRX stays a neutral infrastructure provider across jurisdictions and asset classes. The sealed, append-only record it produces is designed to be *useful evidence to the parties and their own counsel* — not a substitute for them. This is what keeps the primitive domain-agnostic and multi-jurisdiction-ready (multi-jurisdiction packs are **planned**).

The regulator / oversight view

The Authority-Review role is the legal posture made concrete: an oversight party can confirm the *vetted state* of the system without ever seeing private buyer data. Oversight of process, not exposure of parties. This is the same opacity invariant from §4 extended to the regulator surface — the moat does not have a privileged bypass.

SECTION 9

Status & roadmap

Honest status, drawn from the capability list and the audit's deferred-controls section. Built means it runs today; the rest is labelled truthfully.

Built today

- ✓ On-chain reservation / hold / exclusive / attestation registry, live on NEAR testnet.
- ✓ Role-scoped dashboards for all participant types; browse → book → reserve → hold funnel.
- ✓ Attested off-chain records; commission + settlement *projection*.
- ✓ Booking tokens (scarce, cascading, tamper-evident) shown as "coins".
- ✓ S&P contract authoring (payment plans, packages, concessions, termination terms).
- ✓ Vetting / listing gate; change-notice + oversight; off-chain enforcement.
- ✓ $VRX off-chain credit ledger (claim / send / spend).
- ✓ Provenance-stamped imagery with a two-stage reveal (stamping mechanism private).
- ✓ Participation / node-license funnel + a live ecosystem map of the network.

In progress

- Richer agency console (per-agent cards, chat, scheduling/dispatch).
- Real-wallet verified sign-in (NEP-413) — the seam is built, default off.

Planned

- Real on-chain settlement rail; secondary market (ownership transfer); multi-jurisdiction packs.
- $VRX on-chain NEP-141 fungible-token form.
- Determined-attacker-resistant image watermark + the read-side image analyzer.

Deferred-by-design controls (with gating reason)

CONTROL	STATE TODAY	GATING REASON
KYC structured-fields encryption-at-rest	images encrypted; fields plaintext	batched with the pre-mainnet crypto-hardening pass; not a counterparty leak
Authentication (NEP-413)	seam built, flag off	demo is deliberately wallet-less; authz enforced server-side, authn is the phased piece
Off-chain read-scope (catalog tier)	interim caller-scoped guard	hardens with auth; competitive-confidentiality on demo data, not a moat breach
Global seal pepper deployment	supported, empty-safe default	one-function swap at pre-mainnet (env → KMS/HSM)

Path: testnet (`calculating-hand.testnet`) → mainnet. No known privacy/correctness leak is open; the deferred items are deliberate and sequenced behind a dependency.

SECTION 10 · APPENDIX

Contract method surface

The real callable surface, by area (names verbatim from `contract/src/lib.rs`). Writers are `#[payable]` and charge storage + a flat usage toll to non-exempt callers; exempt callers (owner + the exempt list) attach nothing.

INIT & MIGRATION

`new` · `migrate` · `migrate2` … `migrate7` (owner-only, borsh-safe)

NODE / ORG / PEER

`register_node` · `get_node` · `register_org` · `get_org` · `register_peer` · `deactivate_peer` · `get_peer` · `deactivate_node` · `deactivate_org`

ACCESS GATES

`grant_gate` · `revoke_gate` · `update_gate_scope` · `get_gate` · `can_access`

COMMISSION AGREEMENTS

`set_agreement` · `revoke_agreement` · `get_agreement`

ATTESTATIONS

`attest` · `ratify_attestation` · `get_attestation`

TILES (AVAILABILITY)

`stamp_tile` · `stamp_tiles` · `get_tile` · `finalize_tile`

HOLDS & EXCLUSIVES

`release` · `get_hold_expiry` · `set_hold_window` · `get_hold_window` · `grant_exclusive` · `release_exclusive` · `release_exclusive_if_expired` · `get_exclusive`

CLAIMS

`open_claim` · `reply_claim` · `confirm_claim` · `cleanup_claim` · `get_claim` · `get_pending_claims`

BOOKING PASSES

`mint_passes` · `burn_pass_hold` · `rebirth_pass` · `active_pass_count` · `list_passes` · `get_pass` · `get_pass_link`

PROTOCOL FEE / ADMIN

`add_exempt` · `remove_exempt` · `set_toll` · `set_treasury` · `get_toll` · `get_treasury` · `is_exempt` · `required_deposit` · `get_owner`

EVENTS (NEP-297)

State changes emit structured `EVENT_JSON` logs (standard `vrx`, v1.0.0) so indexers can follow without polling. Events carry only data already on-chain — opaque ids, keys, account ids, status, version — never PII, amounts or terms.

SECTION 10 · APPENDIX

Glossary & the demo

Glossary

TERM	MEANING
`node`	A seller / operator (e.g. a developer) on-chain.
`org`	An agency (or a developer's captive in-house sales team).
`peer`	An agent, registered under an org.
`gate`	An access grant from a node to an org over a set of opaque scope hashes.
`tile`	A unit's on-chain stamp; status 0=open 1=hold 2=closed 3=done.
`claim`	A booking request; status 0=open 1=ok 2=no 3=alt 4=confirmed.
`pass`	A non-transferable booking token (reservation capacity); burn → hold, with rebirth.
`exclusive`	A bounded window granting one org the sole right to book a unit.
`attestation`	An on-chain anchor of a sealed sign-off; developer attests, VRX ratifies.
`seal`	A salted SHA-256 over an off-chain record — tamper-evidence without disclosure.
`unit_key` / `request_hash`	Opaque hashes standing in for a unit / a request on-chain.
$VRX	In-system credit (fixed 200M, treasury-held); off-chain ledger today, NEP-141 planned.

Run it

THE REPRODUCIBLE END-TO-END WALK

The contract ships a reproducible end-to-end walk runnable with `npm run demo` (from the portal) — a near-workspaces sandbox that stamps a tile, opens and confirms a claim, and exercises holds against a real in-process NEAR runtime. The live portal demo is at `vrx.poolo.org` — a seeded sandbox: enter wallet-free via `?enter=1`, assume any role (Developer / Agency / Agent / Buyer) from the top bar, and walk one deal end to end. **Write actions in the sandbox are simulated**; the contract on NEAR testnet is the honesty anchor, not the sandbox clicks.

One paragraph, for a protocol reader

VRX turns a public chain's strength — a tamper-evident, independently-verifiable record — into something usable for confidential commerce by writing *only* opaque proof-of-state on-chain and keeping every meaningful detail in a vetted, access-controlled off-chain layer. The contract is a thin, domain-agnostic state machine over hashes; the portal is where authorization, masking and sealing live. The split is the product.

Contact: `vrx@vettedrealty.com` — the business behind the primitive is Vetted Realty. Status language is literal: **built** runs today; **planned** is not yet usable as described.